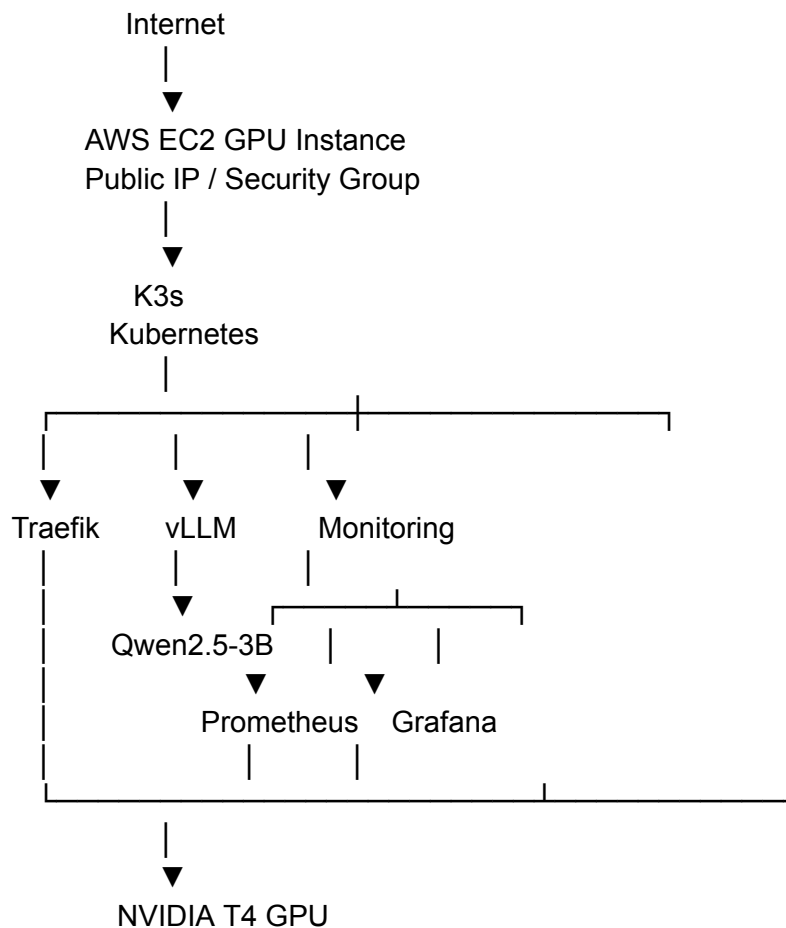


Production-Grade vLLM Deployment on Kubernetes

This project deploys **vLLM with Qwen/Qwen2.5-3B-Instruct** on an NVIDIA GPU-enabled AWS EC2 instance using **K3s Kubernetes**.

The deployment also includes **Prometheus and Grafana** for Kubernetes and application monitoring.

Architecture



1. Prerequisites

Install the following tools:

- Git
- Terraform
- AWS CLI

Verify:

```
terraform --version  
aws --version  
git --version
```

2. Configure AWS CLI

Configure your AWS credentials:

```
aws configure
```

Enter:

```
AWS Access Key ID:  
AWS Secret Access Key:  
Default region name: ap-south-1  
Default output format: json
```

Never commit AWS credentials, access keys, secret keys, or other credentials to GitHub.

3. Verify AWS Account

Verify that the AWS CLI is authenticated:

```
aws sts get-caller-identity
```

Expected output:

```
{
```

```
"UserId": "...",  
"Account": "...",  
"Arn": "..."  
}
```

If this command returns your AWS account information, AWS CLI is configured correctly.

4. Clone the Repository

```
git clone  
https://github.com/UjwalNagrikar/Production-Grade-vLLM-Deployment-on-Kubernetes.git
```

Enter the project:

```
cd Production-Grade-vLLM-Deployment-on-Kubernetes
```

5. Create AWS Infrastructure with Terraform

Go to the Terraform directory:

```
cd terraform
```

Initialize Terraform:

```
terraform init
```

Format Terraform files:

```
terraform fmt
```

Validate the configuration:

```
terraform validate
```

Create the infrastructure:

```
terraform apply
```

Review the plan and type:

yes

Terraform provisions the AWS infrastructure defined in this project.

After the infrastructure is created, connect to the GPU EC2 instance.

6. Connect to the EC2 GPU Instance

Example:

```
ssh -i "your-key.pem" ubuntu@<EC2-PUBLIC-IP>
```

Then:

```
cd ~/Production-Grade-vLLM-Deployment-on-Kubernetes
```

7. Install NVIDIA GPU Components

Run:

```
sudo bash scripts/install-nvidia.sh
```

Verify the GPU:

```
nvidia-smi
```

You should see the NVIDIA GPU information.

Example:

Tesla T4

8. Install K3s Kubernetes

Run:

```
sudo bash scripts/install-k3s.sh
```

Configure kubectl:

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
```

Verify:

```
kubectl get nodes
```

Expected:

NAME	STATUS	ROLES
<node-name>	Ready	control-plane,master

9. Verify NVIDIA GPU in Kubernetes

Check the GPU resource:

```
kubectl get nodes -o  
custom-columns=NAME:.metadata.name,GPU:.status.capacity.nvidia\\.com/gpu
```

Expected:

NAME	GPU
<node-name>	1

Check the NVIDIA device plugin:

```
kubectl get pods -n nvidia-device-plugin
```

The NVIDIA device plugin should show:

Running

10. Deploy vLLM

The vLLM deployment is managed using the Helm chart included in this repository.

From the project root:

```
cd ~/Production-Grade-vLLM-Deployment-on-Kubernetes
```

Deploy vLLM:

```
helm upgrade --install qwen-vllm ./helm/vllm \
  --namespace ai-inference \
  --create-namespace
```

Check the deployment:

```
kubectl get deployment -n ai-inference
```

Check the pod:

```
kubectl get pods -n ai-inference
```

Expected:

NAME	READY	STATUS
qwen-vllm-vllm-xxxxx	1/1	Running

11. Verify vLLM

Check the vLLM logs:

```
kubectl logs -n ai-inference deployment/qwen-vllm-vllm
```

The logs should show:

Qwen/Qwen2.5-3B-Instruct

Check the service:

```
kubectl get svc -n ai-inference
```

12. Test the vLLM API

The deployment exposes an OpenAI-compatible API.

Open the Swagger UI:

<http://<EC2-PUBLIC-IP>/docs>

The model endpoint is:

<http://<EC2-PUBLIC-IP>/v1/models>

You should see:

```
{  
  "id": "Qwen/Qwen2.5-3B-Instruct"  
}
```

13. Test Chat Completion

Open:

<http://<EC2-PUBLIC-IP>/docs>

Find:

POST /v1/chat/completions

Click **Try it out**.

Use:

```
{  
  "model": "Qwen/Qwen2.5-3B-Instruct",  
  "messages": [  
    {  
      "role": "user",  
      "content": "Explain Kubernetes in simple terms."  
    }  
  ],  
  "temperature": 0.7,  
  "max_tokens": 200  
}
```

Click **Execute**.

Successful inference should return:

HTTP 200

with the generated response.

14. Monitoring Setup

This project uses:

Prometheus

+

Grafana

for monitoring Kubernetes and application infrastructure.

The monitoring stack is deployed in the:

monitoring

namespace.

Check the monitoring pods:

```
kubectl get pods -n monitoring
```

Expected components include:

Grafana

Prometheus

Alertmanager

Kube State Metrics

Node Exporter

Prometheus Operator

Example:

```
kube-prometheus-stack-grafana          Running
prometheus-kube-prometheus-stack-prometheus-0 Running
alertmanager-kube-prometheus-stack-alertmanager-0 Running
```

15. Access Grafana

Check the Grafana service:

```
kubectl get svc -n monitoring kube-prometheus-stack-grafana
```

The service should be exposed using a NodePort.

Example:

PORT(S)
80:31293/TCP

The NodePort may be different on another installation.

Use the actual port returned by Kubernetes.

For example:

`http://<EC2-PUBLIC-IP>:31293`

Open this URL in your browser.

16. Grafana Login

Get the Grafana administrator password:

```
kubectl get secret -n monitoring kube-prometheus-stack-grafana \
-o jsonpath="{.data.admin-password}" | base64 -d
echo
```

Default username:

admin

Use the generated password to log in.

17. Configure Prometheus Data Source in Grafana

After logging into Grafana:

Connections



Data sources



Prometheus

The Prometheus data source should point to the Prometheus Kubernetes service.

A typical internal Kubernetes URL is:

`http://kube-prometheus-stack-prometheus.monitoring.svc:9090`

Click:

Save & test

You should receive a successful connection message.

18. Verify Prometheus

Prometheus can also be exposed through a NodePort.

Check:

```
kubectl get svc -n monitoring kube-prometheus-stack-prometheus
```

Example:

`9090:30090/TCP`

The NodePort may be different.

Open:

`http://<EC2-PUBLIC-IP>:30090`

Then navigate to:

Status



Targets

Targets should show their current state.

A healthy target should show:

UP

19. Import a Grafana Dashboard

In Grafana:

Dashboards



New



Import

You can import a dashboard using:

Grafana.com dashboard URL or ID

Enter the desired dashboard ID and click:

Load

When prompted for the data source, select:

Prometheus

Then click:

Import

20. Kubernetes Monitoring Dashboard

A Kubernetes dashboard can display metrics such as:

Node CPU

Node Memory

Pod CPU

Pod Memory

Pod Count

Network Traffic

Container Resources

Kubernetes Workloads

This allows you to monitor the health of the K3s cluster from Grafana.

21. vLLM Monitoring

The vLLM API exposes Prometheus-compatible metrics through:

`/metrics`

The monitoring setup can be used to track metrics such as:

Request rate
Request latency
Token throughput
Running requests
Waiting requests
KV cache usage
GPU utilization
GPU memory
CPU usage
Memory usage

The exact metrics available depend on the deployed vLLM version and monitoring configuration.

22. Verify vLLM Metrics

Check whether the vLLM ServiceMonitor exists:

```
kubectl get servicemonitor -n ai-inference
```

Check Prometheus targets from the Prometheus UI:

Status



Targets

The vLLM target should show:

UP

You can also use Grafana:

Explore



Prometheus

Search for:

vllm

to find available vLLM metrics.

23. Recommended Grafana Dashboard

For this project, the monitoring dashboard should ideally display:

vLLM Kubernetes Monitoring			
Requests/sec	Latency	Error Rate	
Request Rate			
Token Throughput			
GPU Utilization	GPU Memory		
CPU Usage	Memory Usage		
Pod Status / Restarts / Kubernetes Health			

24. Useful Kubernetes Commands

All resources

```
kubectl get all -A
```

Nodes

```
kubectl get nodes -o wide
```

GPU

```
kubectl get nodes -o  
custom-columns=NAME:.metadata.name,GPU:.status.capacity.nvidia\\.com/gpu
```

vLLM pods

```
kubectl get pods -n ai-inference
```

Monitoring pods

```
kubectl get pods -n monitoring
```

NVIDIA device plugin

```
kubectl get pods -n nvidia-device-plugin
```

Services

```
kubectl get svc -A
```

ServiceMonitors

```
kubectl get servicemonitor -A
```

Detailed pod information

```
kubectl describe pod <pod-name> -n <namespace>
```

vLLM logs

```
kubectl logs -n ai-inference deployment/qwen-vllm-vllm
```

Follow vLLM logs

```
kubectl logs -f -n ai-inference deployment/qwen-vllm-vllm
```

25. Troubleshooting

Kubernetes is not responding

```
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml  
kubectl get nodes
```

GPU is not available

Check the NVIDIA driver:

```
nvidia-smi
```

Then:

```
kubectl get nodes -o  
custom-columns=NAME:.metadata.name,GPU:.status.capacity.nvidia\\.com/gpu
```

vLLM pod is not running

```
kubectl get pods -n ai-inference
```

Then:

```
kubectl describe pod <pod-name> -n ai-inference
```

Check logs:

```
kubectl logs -n ai-inference deployment/qwen-vllm-vllm
```

Grafana is not accessible

Check:

```
kubectl get svc -n monitoring kube-prometheus-stack-grafana
```

Then check the NodePort.

For example:

80:31293/TCP

Access:

<http://<EC2-PUBLIC-IP>:31293>

Also make sure the corresponding TCP NodePort is allowed by the EC2 Security Group.

Prometheus target is DOWN

Check:

```
kubectl get servicemonitor -A
```

Then inspect Prometheus:

Prometheus

→ Status

→ Targets

Check the target's error message and verify that the corresponding Kubernetes Service and `/metrics` endpoint are available.

26. Security

Never commit the following to GitHub:

AWS access keys

AWS secret keys

GitHub tokens

SSH private keys

*.pem

.env files

Kubernetes credentials

Passwords

API keys

Private certificates

Before committing:

git status

Review all changed files carefully.

27. Complete Deployment Flow

For a new user, the overall process is:

1. Install Git

↓

2. Install AWS CLI

↓

3. Install Terraform

↓

4. aws configure
 - ↓
 5. aws sts get-caller-identity
 - ↓
 6. Clone GitHub repository
 - ↓
 7. terraform init
 - ↓
 8. terraform fmt
 - ↓
 9. terraform validate
 - ↓
 10. terraform apply
 - ↓
 11. SSH into GPU EC2
 - ↓
 12. install NVIDIA
 - ↓
 13. install K3s
 - ↓
 14. Verify GPU
 - ↓
 15. Deploy vLLM
 - ↓
 16. Verify Qwen2.5-3B
 - ↓
 17. Deploy/verify Prometheus
 - ↓
 18. Open Grafana
 - ↓
 19. Configure Prometheus datasource
 - ↓
 20. Import Kubernetes dashboard
 - ↓
 21. Verify vLLM metrics
 - ↓
 22. Test API inference
-

28. Final Verification

Run:

```
kubectl get nodes -o wide  
kubectl get pods -n ai-inference  
kubectl get pods -n monitoring  
kubectl get pods -n nvidia-device-plugin  
kubectl get svc -A
```

Verify:

Kubernetes	→ Ready
NVIDIA GPU	→ Available
NVIDIA Plugin	→ Running
vLLM	→ Running
Qwen Model	→ Loaded
Prometheus	→ Running
Grafana	→ Running
Monitoring	→ Working
API Inference	→ HTTP 200

Project Repository

GitHub:

[Production-Grade-vLLM-Deployment-on-Kubernetes](#)